# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

MCS 481 Lecture 25
Computational Geometry
Jan Verschelde, 21 October 2026

# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# SageMath and CoCalc

- SageMath is a free and open source software system for mathematical computations, using Python as its language.
- It is available in the cloud, at `cocalc.com`.
- An example from the SageMath documentation:

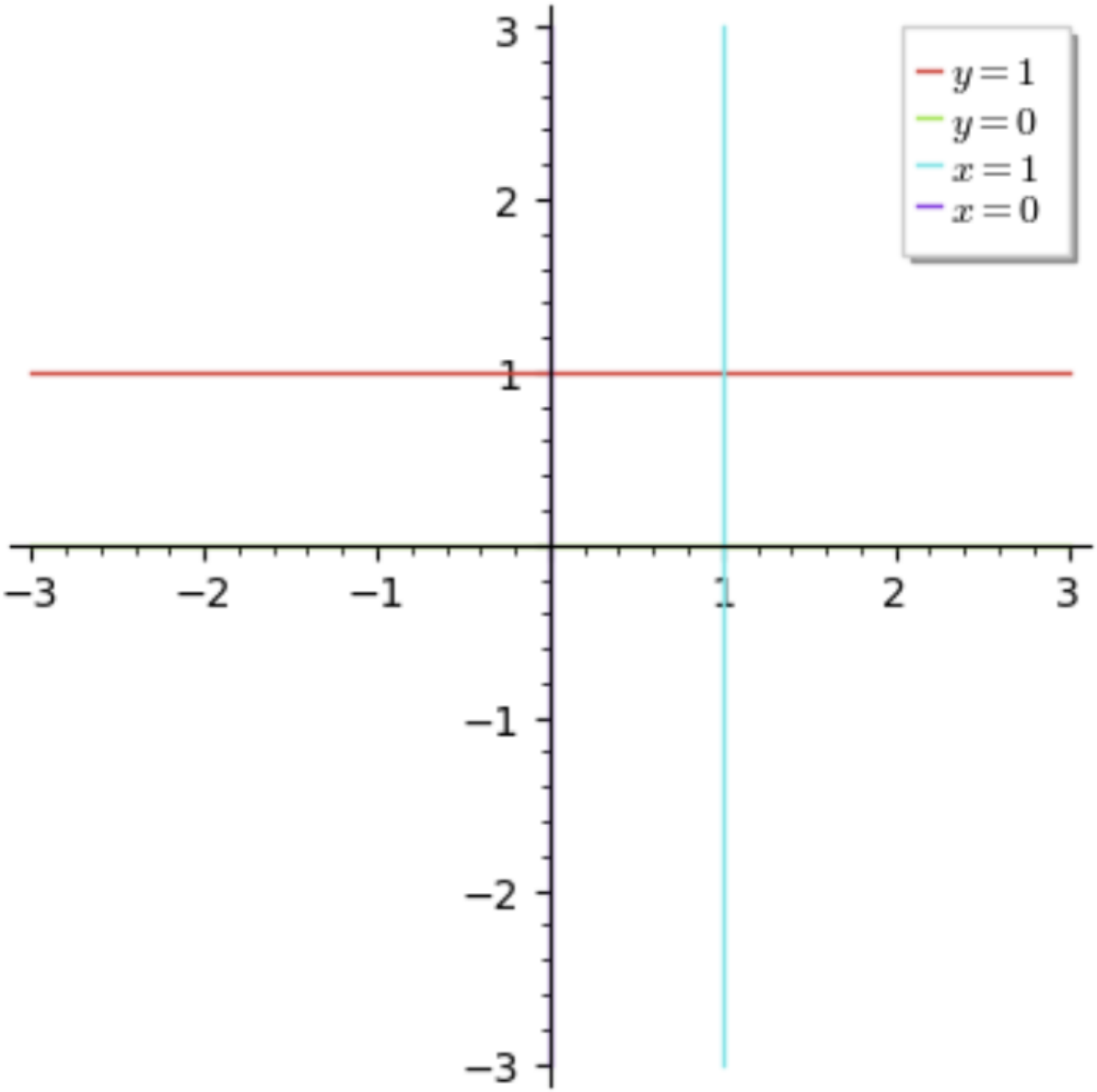
```
H.<x,y> = HyperplaneArrangements(QQ)
box = x | y | x-1 | y-1
box.plot()
```

The `QQ` is the field of rational numbers $\mathbb{Q}$.

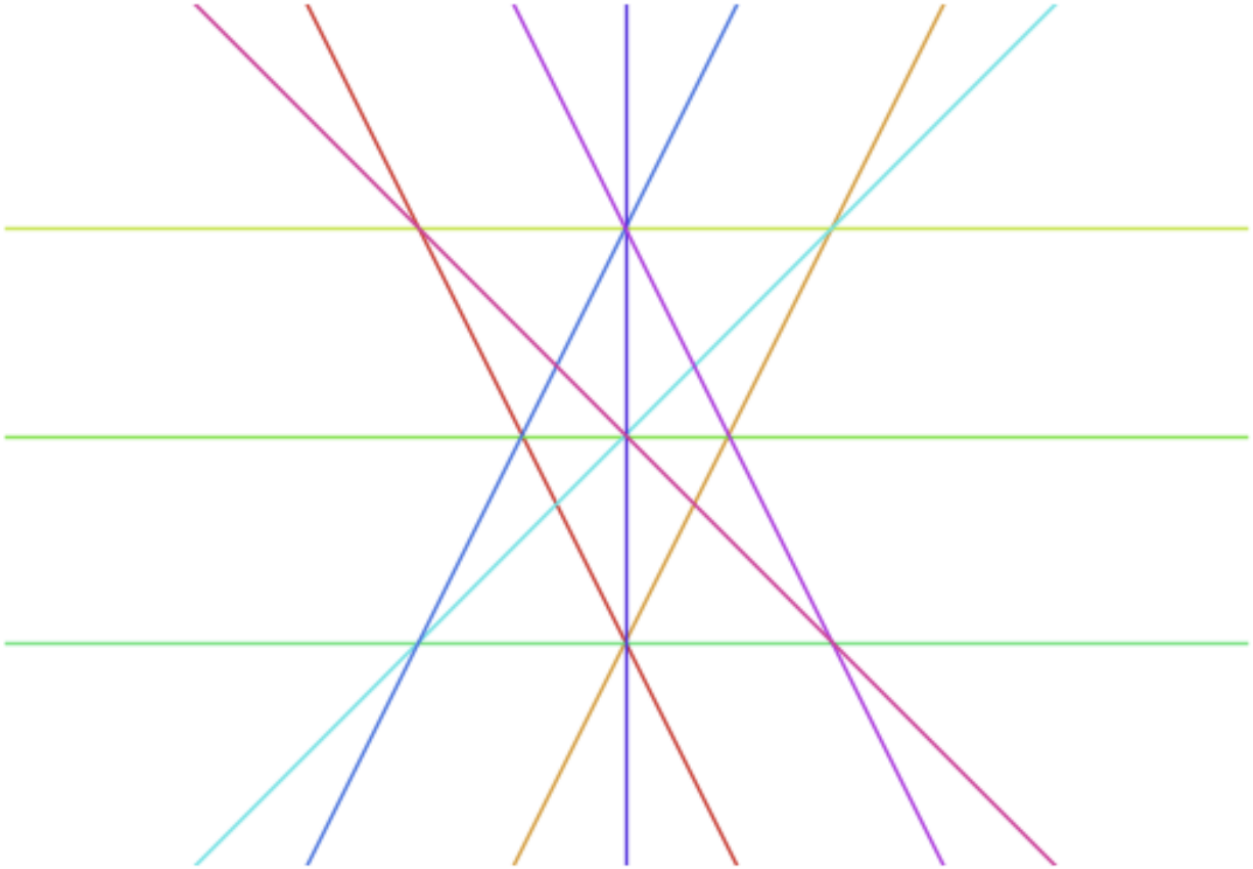
Equations are defined using the symbols `x` and `y`.

See the posted Jupyter notebook.

# a first example from the SageMath documentation

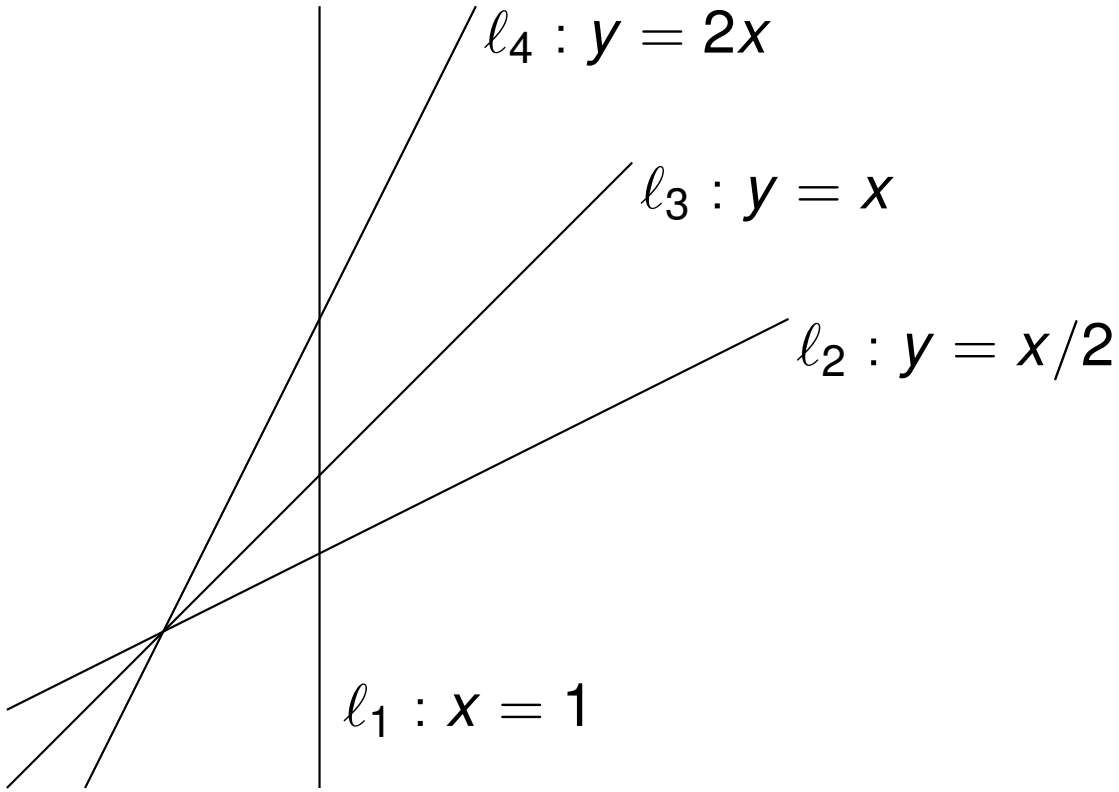


# three rows of nine points: an orchard



# a first exercise

Exercise 1: Consider the four lines $\ell_1$, $\ell_2$, $\ell_3$, $\ell_4$ as defined below.

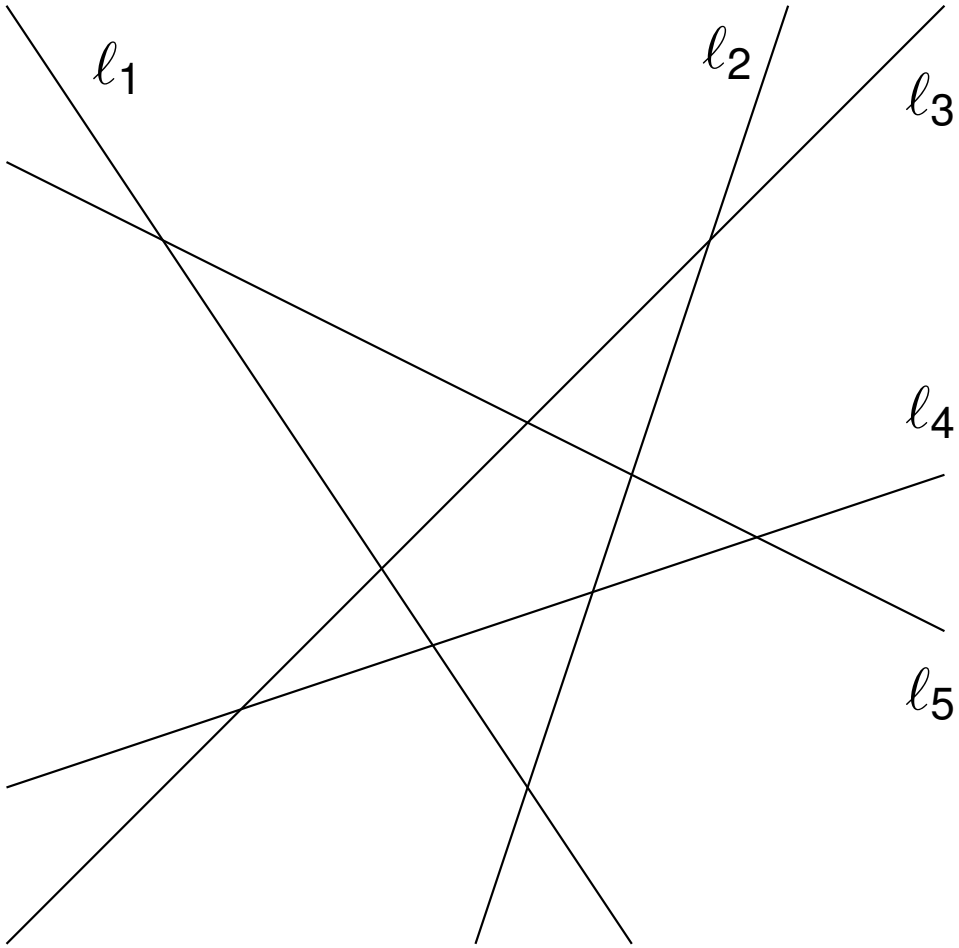



1. Define this arrangement of lines with SageMath.
2. Count the number of vertices and faces.

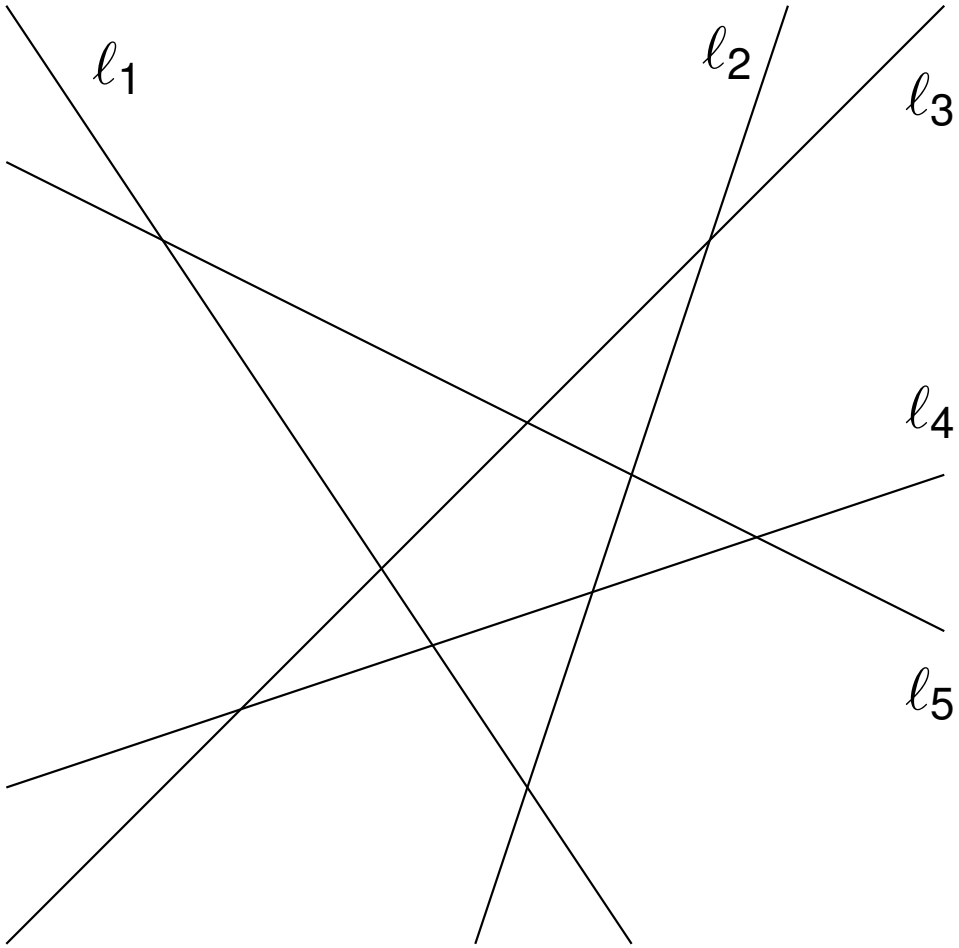
# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# five lines in the plane



# five lines in the plane induce a subdivision




10 vertices

25 edges

16 faces

# definitions

## Definition (arrangement of lines)

Let $L$ be a set of $n$ lines in the plane.
An *arrangement $A(L)$ of lines* is a planar subdivision consisting of

1. vertices, the intersections of the lines,
2. edges, the segments of lines between two vertices, and
3. faces, the regions bounded by the edges.

## Definition (simple arrangement)

An arrangement of lines is *simple* if

1. no three lines pass through the same point, and
2. no two lines are parallel.

# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# combinatorial complexity

The complexitiy of an arrangement of $n$ lines is $O(n^2)$.

**Theorem (combinatorial complexity of arrangement of lines)**

*Let $A(L)$ be an arrangement of lines, $n = \#L$.*

1. *The number of vertices $\leq n(n-1)/2$.*
2. *The number of edges $\leq n^2$.*
3. *The number of faces $\leq n^2/2 + n/2 + 1$.*

*Equality of all bounds $\Leftrightarrow$ $A(L)$ is simple.*

The three statements in the theorem will be implied by three lemmas on the next slides.

# the number of vertices

## Lemma (the number of vertices in an arrangement)

*The number of vertices in the arrangement $A(L)$, $n = \#L$,*

1. *is bounded by $n(n-1)/2$, and moreover,*
2. *is equal to $n(n-1)/2 \Leftrightarrow A(L)$ is simple.*

*Proof.*

1. We have $\binom{n}{2}$ pairs of lines,
   thus as most $n(n-1)/2$ intersections.
2. If we have exactly $n(n-1)/2$ intersections, then
   1. no three lines can pass through the same point, and
   2. no two lines are parallel,

   so $A(L)$ is simple.

   If $A(L)$ is simple,
   then we will have exactly $n(n-1)/2$ intersections.

# the number of edges

## Lemma (the number of edges in an arrangement)

*The number of edges in the arrangement $A(L)$, $n = \#L$,*

1. *is bounded by $n^2$, and moreover,*
2. *is equal to $n^2 \Leftrightarrow A(L)$ is simple.*

*Proof.*

1. Every line is intersected by at most $n-1$ lines, so the number of edges on one line is at most $n$, and the total number of edges is bounded by $n^2$.
2. If we have exactly $n^2$ edges, then every line is intersected by exactly $n-1$ lines and $A(L)$ is simple.

   If $A(L)$ is simple, then every edge is intersected by exactly $n-1$ lines and has exactly $n$ edges.

# the number of faces

## Lemma (the number of faces in an arrangement)

*The number of faces in the arrangement $A(L)$, $n = \#L$,*

1. *is bounded by $n^2/2 + n/2 + 1$, and moreover,*
2. *is equal to $n^2/2 + n/2 + 1 \Leftrightarrow A(L)$ is simple.*

*Proof.* Consider adding the lines one after each other.

Adding line $\ell_i$

1. leads to the addition of $i$ edges,
2. and every edge splits a face in two.

We start with one face, so the total number of faces is

$$1 + \sum_{i=1}^{n} i = 1 + n^2/2 + n/2.$$

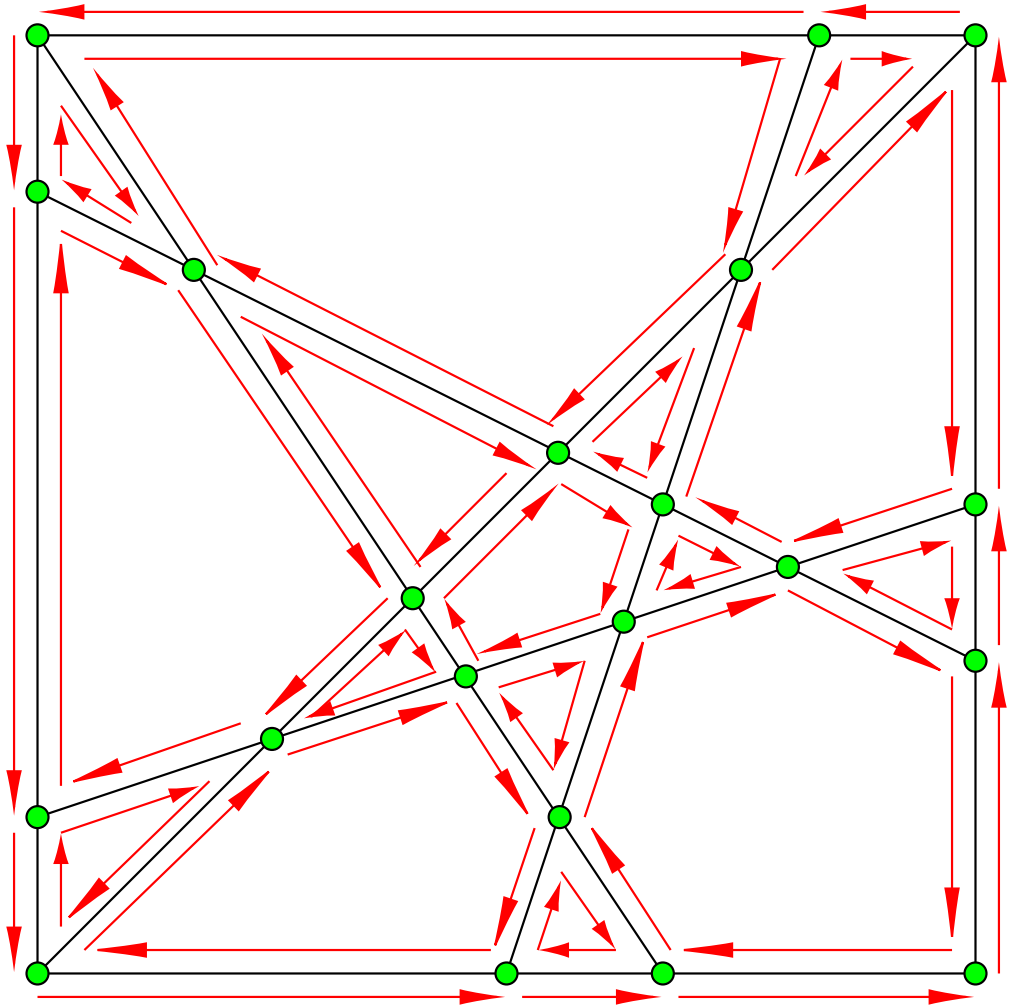
We can have only fewer faces if $A(L)$ is not simple.

Q.E.D.

# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# a doubly connected edge list in a bounding box



Exercise 2: Count the number of above vertices, edges, and faces. Compare the numbers to the counts before the box was added. What are the counts for $n$ general lines in a box?

# the line segment intersection problem

By adding the bounding box,
the given lines turned into line segments.

The line segment intersection problem solves our problem.

We discussed an output sensitive algorithm FINDINTERSECTIONS.

**Lemma (running time of FINDINTERSECTIONS)**

*For $n$ segments,* FINDINTERSECTIONS *runs in $O((n+I)\log(n))$ time, where $I$ is the number of intersection points.*

The number $I$ of intersection points is $n^2$ for simple $A(L)$,
so we arrive at $O(n^2 \log(n))$, which is not optimal.

# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# an incremental algorithm

There are two main steps:

1. Compute all intersection points first.

   Consider the leftmost, rightmost, top and bottom vertex to define the bounding box, so the box contains those four vertices.

   This step takes $O(n^2)$ running time.

2. Add the lines $\ell_1, \ell_2, \ldots, \ell_n$ one after the other, adding edges and faces to the doubly connected edge list.

   This step will be elaborated further.

# on the size of the bounding box

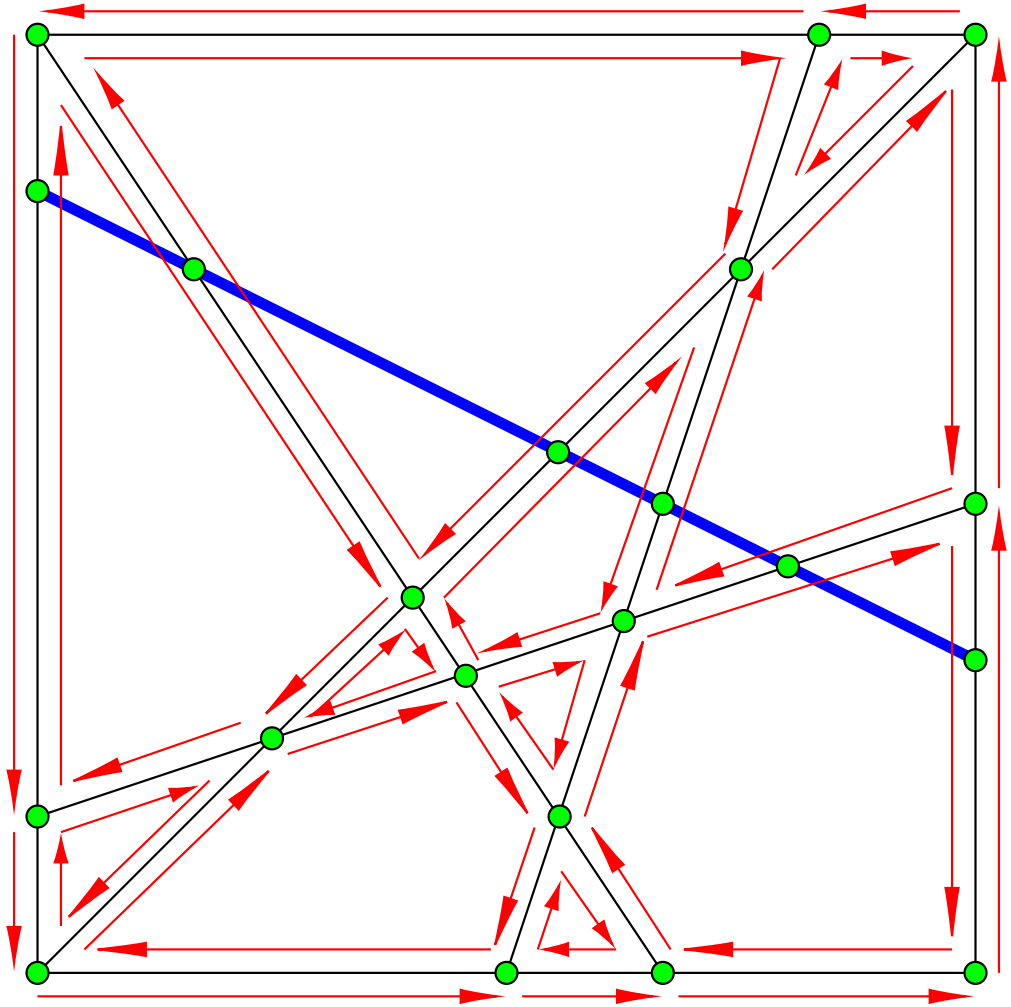
Exercise 3: The bounding box should contain all vertices.

1. What does the relative position of the lines imply for the size of the bounding box?
   1. Give an example of a worst case line arrangement.
   2. Give an example of a best case line arrangement.
2. In numerical analysis, the best condition number occurs when the matrix of a linear system is orthogonal.

   Specific for the problem of intersecting lines: when the lines are perpendicular to each other.

   Can you define a condition number for a line arrangement?

# adding a new line



# adding a new line

Let $A_i$ denote the subdivision of the plane

- induced by the bounding box $B(L)$, and
- the part of $A(\{\ell_1, \ell_2, \ldots, \ell_i\})$ inside $B(L)$.

To add a new line $\ell_i$,
we must split the faces in $A_{i-1}$ that are intersected by $\ell_i$.

We have already computed all the intersection points on $\ell_i$,
order those intersection points from left to right.

# walking on the new line from left to right

We have already computed all the intersection points on $\ell_i$,
order those intersection points from left to right.

In the walk along the line $\ell_i$ from left to right

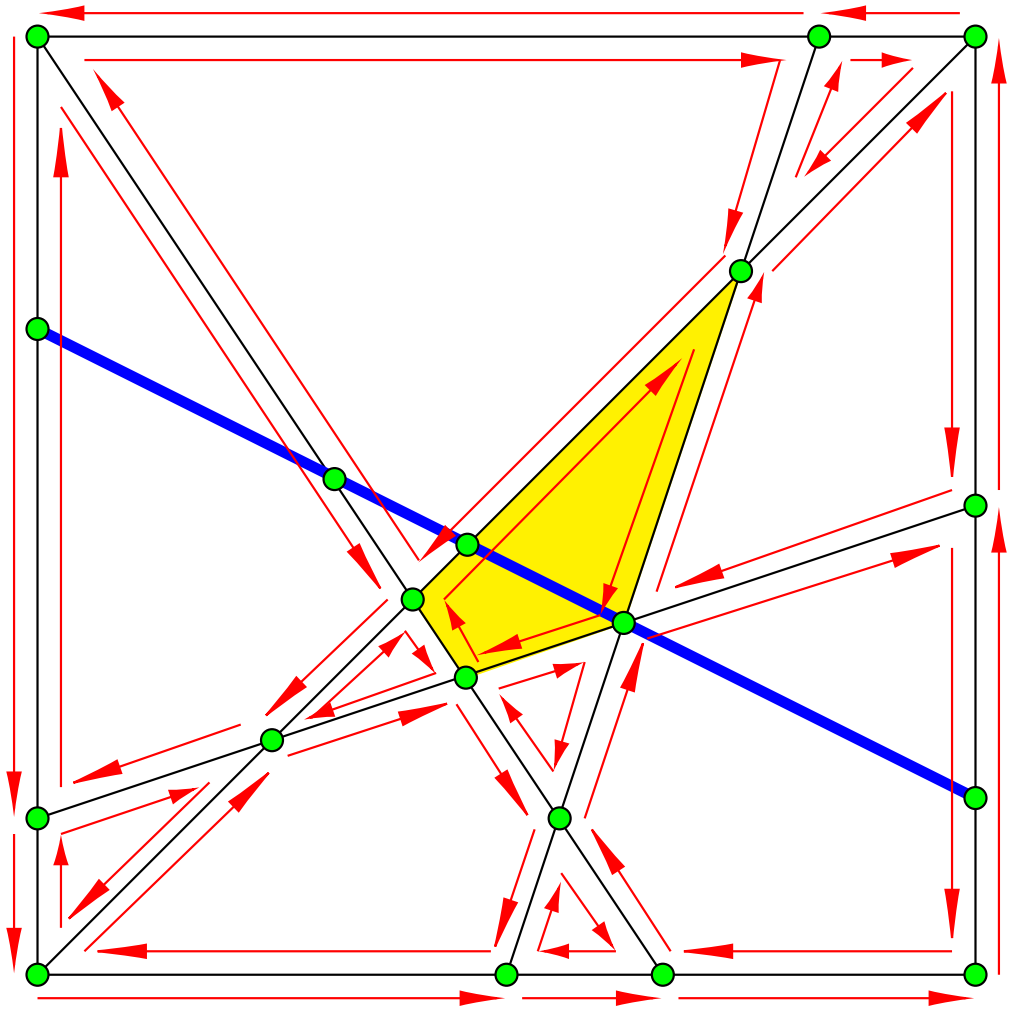
1. enter the face $f$ through the edge $e$,
   follow the NEXT() pointer to the edge $e'$ where $\ell_i$ leaves $f$,
2. go to TWIN($e'$) to get to the adjacent face.

If we leave a face at an existing vertex $v$,
then walk around $v$ until the next face intersected by $\ell_i$ is found.

Two issues left to consider:

1. How to find the leftmost edge intersected by $\ell_i$?
2. How to split the faces we encounter?

# the new line leaves a face through a vertex



Exercise 4: Consider the face $f$ where the new line leaves through a vertex. Describe the finding of the face adjacent to $f$.

# finding the leftmost edge

Where is the leftmost edge of $A_{i-1} \cap \ell_i$?

How many edges does $A_{i-1}$ contain on $B(L)$ at most?

$$\rightarrow A_{i-1} \text{ has at most } 2i + 2 \text{ edges on } B(L).$$

To find the leftmost edge,
we test all edges of $A_{i-1}$ on $B(L)$ in $O(i)$ time
for intersection with $\ell_i$.

# splitting a face intersected by the new line

Let $f$ be a face intersected by $\ell_i$.
Assume the edge at the left of $f$ is already split.

1. Make two new face records, one above $\ell_i$, one below $\ell_i$.
2. Split $e'$ where $\ell_i$ leaves the face, make a new vertex on $e'$.
3. Add two new half edges along $\ell_i$ through $f$.
4. Update pointers to IncidentFace.
5. Destroy old record for $f$ and the half edge recordes for $e'$.

The total time for the split is linear in the complexity of $f$.

# Arrangements of Lines

1. Computing Arrangements with SageMath
   - generating examples
2. Arrangements of Lines
   - examples and definitions
   - combinatorial complexity
3. An Incremental Algorithm
   - a doubly connected edge list in a bounding box
   - adding lines one after the other
   - formulation of the algorithm

# the algorithm

Algorithm ConstructArrangement($L$)

Input: a set $L$ of $n$ lines.
Output: $A(L)$, stored in doubly connected edge list,
within a bounding box $B(L)$.

1. compute $B(L)$ enclosing all vertices of $A(L)$
2. construct a doubly connected edge list $\mathcal{D}$ to store $B(L)$
3. for $i$ from 1 to $n$ do
4. find the edge $e$ on $B(L)$
that contains leftmost intersection point of $\ell_i$ and $A_{i-1}$
5. let $f$ be the bounded face incident to $e$
6. while $f$ is not outside $B(L)$ do
7. split $f$, update $\mathcal{D}$
8. set $f$ to the next face intersected by $\ell_i$

# introducing the zone theorem

In the next lecture we will apply arrangements to the discrepancy computation and prove the zone theorem.

The zone of a line in an arrangement is the closure of all faces of the arrangement intersected by the line.

**Theorem (the zone theorem)**

*Let $L$ be a set of $m$ lines and $\ell$ be some line.*
*The zone complexity of a line $\ell$ in the arrangement $A(L)$ is $O(m)$.*

By the zone theorem,
the cost of the incremental algorithm is quadratic.

# summary and exercises

We started section 8.3 in the textbook.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, 3, and 4.
2. Consult the CGAL documentation and example code on arrangements of lines.
3. Consider the exercises 5, 6, 9 in the textbook.